

排班系統前端技術教學

Vue 3、TypeScript、Router、Pinia 與 Axios 實作導讀

這份教材特別把 router、元件、狀態與 API 串接拆開說明，適合前端基礎還不熟時循序閱讀。

1. 前端在做什麼？

前端是瀏覽器裡的應用程式。Vue 負責把資料轉成畫面，Router 決定 URL 對應哪個頁面，Pinia 保存跨頁共用的登入狀態，Axios 負責呼叫後端 API。

檔案	用途
main.ts	建立 Vue app、掛上 Pinia 與 router
App.vue	最外層版型與 router-view
router.ts	URL、頁面與登入導頁規則
stores/auth.ts	login、logout、目前 user
api.ts	Axios baseURL、Authorization、錯誤轉換
views/*.vue	實際畫面：登入、員工班表、老闆總覽

2. Vue 3 元件與 reactive data

每個 .vue 檔通常由 template、script setup、style 三部分組成。template 描述畫面，script setup 寫資料與事件，style 管外觀。ref() 建立可變 reactive value，讀取時用 .value；在 template 中 Vue 會自動解開。

```
<script setup lang="ts">
import { ref } from "vue"
const count = ref(0)
function add() { count.value++ }
</script>
<template><button @click="add">{{ count }}</button></template>
```

常見語法：v-if 控制是否顯示，v-for 產生清單，:class 綁定 class，@click 綁定事件，v-model 雙向綁定輸入框。

3. Router 詳解：URL 如何切換頁面

createRouter() 建立 router；createWebHistory() 使用瀏覽器的正常 URL。routes 是路由表，每一筆 route 把 path 對應到 component。App.vue 通常放，匹配到的頁面會渲染在那裡。

```
const router = createRouter({
  history: createWebHistory(),
  routes: [
    { path: "/login", component: LoginView },
    { path: "/schedule", component: ScheduleView,
      meta: { role: "EMPLOYEE" } },
    { path: "/dashboard", component: DashboardView,
      meta: { role: "OWNER" } },
  ],
})
```

path 是網址；component 是頁面；meta

是你自己附加的資料，這裡用來註記該頁需要的角色。路由本身不會自動阻擋使用者，必須搭配 navigation guard。

步驟	發生什麼事
1	LoginView 取得 email、password
2	auth.login() 呼叫 POST /api/auth/login
3	後端回傳 access token，並設定 HttpOnly refresh cookie
4	store 只在記憶體保存 access token 與 user
5	頁面啟動時 router guard 先 await auth.initialize()
6	access token 過期時 Axios 自動 refresh 並重送原請求

除錯方式：Network 看 login、refresh、原 API 的 status code；不要期待在 Application > Local Storage 找到 token。可檢查 Cookie 是否存在，以及 request 是否帶 Authorization。

8. 你可以怎麼修改這個前端？

新增頁面：建立 views/ReportView.vue → 在 router.ts 加 route → 若需要登入，在 meta 放 role → 由 App.vue 的 router-view 顯示。新增 API：在 view 或獨立 composable 呼叫 api.get/post → 用 ref 保存 loading、data、error → template 依狀態顯示。

```
const loading = ref(false)
const error = ref("")
async function load() {
  loading.value = true
  try { data.value = (await api.get("/reports")).data }
  catch (e) { error.value = (e as Error).message }
  finally { loading.value = false }
}
```

練習順序：先改文字與樣式，再新增一個 route，接著新增一個 GET API，最後再做表單與錯誤狀態。每次只改一個層次，比較容易定位問題。